

Hybrid Trek Recommendation System

Complete Study Guide -- Final Year Project

ALS Matrix Factorization + Content-Based Filtering
Adaptive Hybrid Blending + Post-Ranking Re-Scorer

Covers: Algorithm theory, Code walk-through,
Interview Q&A, API design, Evaluation metrics

Nepal Trek Dataset | 188 Treks | 600 Synthetic Users

Python | NumPy | Flask | scikit-learn | pandas

Table of Contents

- Section 1** Project Overview & Architecture
- Section 2** Dataset & Feature Engineering
- Section 3** Synthetic User Generation
- Section 4** Interaction Scoring & Weighting
- Section 5** The ALS Algorithm -- Deep Dive
- Section 6** Content-Based Filtering -- Deep Dive
- Section 7** Hybrid Blending Strategy
- Section 8** Post-Ranking Re-Scorer
- Section 9** Evaluation Metrics (RMSE, NDCG, Precision, Recall)
- Section 10** train.py -- Step-by-Step Code Walk-through
- Section 11** app.py -- Flask API Walk-through
- Section 12** Cold-Start Problem & How It Is Solved
- Section 13** Hyperparameter Tuning
- Section 14** Things to Know & Learn
- Section 15** Interview Questions & Model Answers

Section 1 -- Project Overview & Architecture

This project is a Hybrid Trek Recommendation System for Nepal treks. Given a user with certain preferences (difficulty, budget, fitness level, etc.), the system recommends the most suitable treks from a catalog of 188 Nepal treks. The system combines two recommendation paradigms into one pipeline:

- Collaborative Filtering via ALS (Alternating Least Squares) -- learns from how users interact with treks (views, bookings, ratings, favourites, time spent).
- Content-Based Filtering (CBF) -- computes cosine similarity between a user preference vector and trek feature vectors (24 features per trek).
- Post-Ranking Re-Scorer -- applies small bonuses/penalties based on season suitability, AMS risk, accommodation match, and permit willingness.

High-Level Pipeline

The end-to-end pipeline is:

1. Load 188 Nepal trek records from JSON dataset.
2. Engineer 24 numerical features per trek (encode categorical fields, normalise to [0,1]).
3. Synthesize 6 metadata groups per trek: seasonality, AMS detail, permits, accommodation, transport, health & safety.
4. Generate 600 synthetic users across 15 preference profiles.
5. Simulate ~121,000 user-trek interactions with weighted engagement signals.
6. Split 80/20 into train/test sets.
7. Train ALS model (from scratch, no external library) using the training interaction matrix.
8. Build CBF using weighted cosine similarity on the 24-feature space.
9. Combine ALS + CBF with an adaptive alpha blending weight.
10. Apply a lightweight post-ranking bonus using enhanced metadata.
11. Save the full model to models/recommender.pkl.
12. Serve recommendations via a Flask REST API (app.py).

File Map

[train.py](#)

Full training pipeline: data prep, ALS, CBF, hybrid, evaluation, save model

[app.py](#)

Section 2 -- Dataset & Feature Engineering

2.1 Raw Dataset Fields (per trek)

Each of the 188 trek records in the JSON contains the following fields. These are the RAW fields before encoding:

- trek_id, name, location, region, country
- difficulty (very easy / easy / moderate / moderate to strenuous / challenging / very challenging)
- duration_days, distance_km, max_altitude_m, altitude_gain_m, base_camp_altitude_m
- temperature_min, temperature_max (degrees Celsius)
- fitness_level_required (basic / moderate / good / excellent)
- risk_level (very low / low / moderate / high / very high)
- altitude_sickness_risk (none / low / moderate / high / very high)
- water_availability, food_availability (very limited / limited / moderate / good / excellent)
- mobile_network (none / very limited / limited / moderate / good / excellent)
- permits_required (bool), guide_mandatory (bool), evacuation_possible (bool)
- daily_trek_hours, estimated_cost_min_usd, estimated_cost_max_usd
- nearest_medical_facility_km, average_rating, popularity_score
- group_size_min, group_size_max
- terrain_types, best_seasons, attractions, itinerary, province

2.2 The 24-Feature Matrix (what the model actually uses)

Categorical string fields are mapped to integers, then the whole matrix is scaled to [0, 1] with MinMaxScaler. The 24 columns are:

- difficulty -- 1=very easy ... 6=very challenging
- fitness -- 1=basic ... 4=excellent
- risk -- 1=very low ... 5=very high
- altitude_risk -- 0=none ... 4=very high
- water -- 1=very limited ... 5=excellent
- food -- 1=very limited ... 5=excellent
- network -- 1=none ... 6=excellent
- permits -- 0 or 1 (boolean)
- guide -- 0 or 1 (boolean)
- evacuation -- 0 or 1 (boolean)
- duration_days, distance_km, max_altitude_m, altitude_gain_m
- daily_hours, cost_min, cost_max
- temp_min, temp_max, nearest_medical_km
- avg_rating, popularity, group_min, group_max

2.3 Feature Importance Weights

Feature importance is computed as variance-based: features that vary more across treks carry more weight in CBF similarity. Formula:

```
feature_variance = trek_feature_scaled.var(axis=0)
feat_imp_weights = 0.5 + (feature_variance / (feature_variance.max() + 1e-10))
```

Result: weights in [0.5, 1.5] -- high-variance features matter more

2.4 Synthesized Enhanced Metadata (6 groups)

Six extra attribute groups are synthesized from existing trek data. These are NOT used in the ALS/CBF matrices; they are only used in the post-ranking re-scorer:

- seasonality -- spring/summer/autumn/winter ratings (1?5), derived from altitude & temperature
- altitude_sickness -- risk_level, acclimatization_days, highest_pass_m
- permit_details -- types (TIMS / national_park / restricted_area), total_cost_usd, advance_booking_days
- accommodation -- types available (teahouse/lodge/camping/homestay), quality_rating
- transportation -- drive_hours_from_ktm, flight_required
- health_safety -- medical_posts_on_route, helicopter_evac, oxygen_available

Section 3 -- Synthetic User Generation

Because no real user interaction data exists for Nepal treks, 600 synthetic users are generated from 15 predefined "personas". Each persona captures a realistic trekker type, and 40 users per persona are created with small random noise to simulate natural variation.

3.1 The 15 User Profiles

- Beginner Budget -- diff 1, budget \$300, 7 days, fitness 1, autumn, AMS concern 5
- Casual Explorer -- diff 2, budget \$500, 10 days, fitness 2, spring, AMS concern 4
- Weekend Trekker -- diff 2, budget \$400, 5 days, Kathmandu Valley, teahouse
- Nature Lover -- diff 3, budget \$700, 12 days, Annapurna, homestay
- Photography Enthusiast -- diff 3, budget \$800, 14 days, autumn, teahouse
- Cultural Trekker -- diff 2, budget \$600, 10 days, Mustang, homestay
- Family Trekker -- diff 2, budget \$1000, 10 days, lodge
- Fitness Enthusiast -- diff 4, budget \$1000, 14 days, Manaslu, spring
- Adventure Seeker -- diff 5, budget \$2000, 18 days, Khumbu, camping, AMS concern 1
- High Altitude Lover -- diff 5, budget \$2500, 21 days, Khumbu, spring, camping
- Budget Backpacker -- diff 3, budget \$400, 14 days, summer, no permit preference
- Luxury Trekker -- diff 3, budget \$5000, 14 days, Annapurna, lodge
- Solo Trekker -- diff 4, budget \$1500, 16 days, spring, teahouse
- Remote Explorer -- diff 5, budget \$3000, 20 days, Dolpo, camping
- Himalayan Enthusiast -- diff 6, budget \$4000, 25 days, Dhaulagiri, camping, AMS concern 1

3.2 Noise Injection

Each of the 40 users per profile has small noise added so they are not identical:

- difficulty $\pm$ random choice of [-1, 0, +1]
- budget $\times$ uniform random in [0.75, 1.35]
- duration $\pm$ random choice of [-2, -1, 0, +1, +2]
- fitness $\pm$ random choice of [-1, 0, +1]
- AMS concern $\pm$ random choice of [-1, 0, 0, +1]
- 20% chance of random preferred_season (otherwise inherits profile season)
- 15% chance of random accommodation preference

3.3 User Preference Fields

difficulty	1?6 (matches the difficulty scale of treks)
------------	---

budget_m

Section 4 -- Interaction Scoring & Weighting

4.1 Why Interactions?

ALS is a collaborative filtering algorithm -- it works on a USER x ITEM matrix where each cell represents how much a user "liked" an item. Real data would have this naturally. Since we have synthetic users, we simulate interactions using a scoring function that reflects how well a trek matches a user.

4.2 Base Score Computation

The function `compute_base_score(user, trek)` produces a 0-5 score for every (user, trek) pair. It uses weighted sub-scores:

Difficulty match	3.0 -- most important; gap between user difficulty pref and trek difficulty
-------------------------	---

Budget ma

Final score = (weighted sum / total possible weight) × 5.0

4.3 Interaction Signals

45% of all (user, trek) pairs get a simulated interaction record. Each interaction has these fields:

- views -- 0-10, derived as `base_score × 2 + noise`
- booked -- True if `base_score > 4.2` and `random < 0.3`
- favorites -- True if `base_score > 3.5` and `random < 0.4`
- rating -- float 1.0-5.0 if `base_score > 3.0`, else 0.0
- time_spent_secs -- 0-600 seconds, proportional to `base_score`

4.4 Weight Formula (raw ? normalised)

Each interaction record is converted to a single weight value:

```
weight_raw = views * 0.3
            + booked * 3.0
            + favorites * 1.5
            + (rating / 5.0) * 2.0
            + (time_spent_seconds / 600.0) * 1.0

# Min-Max normalise to [0.5, 5.0]
weight = 0.5 + (weight_raw - min_w) * (5.0 - 0.5) / (max_w - min_w)
# + small Gaussian noise (std=0.1), then clipped to [0.5, 5.0]
```

This weight becomes the value in the user-trek interaction matrix R. Higher weight = stronger preference

signal.

Section 5 -- ALS Algorithm -- Deep Dive

5.1 What Is ALS?

ALS stands for Alternating Least Squares. It is a Matrix Factorization technique used in Collaborative Filtering. The central idea is:

Given a sparse interaction matrix R of shape $(n_users \times n_items)$, factorize it into two low-rank matrices:

$$R \approx U \times V^T$$

where:

U = user embedding matrix $(n_users \times n_factors)$

V = item embedding matrix $(n_items \times n_factors)$

Each row of U is a latent vector representing a user.

Each row of V is a latent vector representing an item (trek).

The predicted score for (user u , item i) = $U[u] \cdot V[i]$ (dot product)

5.2 Why Confidence-Weighted ALS (Implicit ALS)?

Standard ALS assumes explicit ratings (e.g. 1-5 stars from the user). Here the "weight" values are implicit signals (views, bookings, time spent). We use the Hu, Koren & Volinsky (2008) formulation where a confidence value c_{ui} is assigned to each (user, item) pair:

```
c_ui = 1 + confidence_scale * w_ui
# confidence_scale is a hyperparameter (tuned to 5.0 or 10.0)
# w_ui is the interaction weight
```

High-weight interactions (e.g. booked + rated) get high confidence ? the model is more penalised for getting those predictions wrong.

5.3 The Objective Function

The loss function ALS minimises is:

```
L = sum over all (u,i) pairs where R[u,i] != 0:
    c_ui * (w_ui - U[u] @ V[i])^2
+ lambda * (||U||^2 + ||V||^2)

# lambda = regularisation strength (prevents overfitting)
# c_ui = confidence weight
# w_ui = observed interaction weight
# U[u]@V[i] = predicted score
```

5.4 Why "Alternating" Least Squares?

The trick: if you hold V fixed and optimise U , the problem becomes a simple ridge regression (closed-form solution). Then hold U fixed and optimise V -- again ridge regression. Alternate between them until convergence.

5.5 User Update Step (closed-form)

For each user u , with V fixed:

```
V_u = V[item_indices_for_u]          # items this user interacted with
conf = 1 + confidence_scale * w      # confidence vector

# A = V^T V + V_u^T diag(conf-1) V_u + lambda * I
A = V.T @ V + V_u.T @ (V_u * (conf-1)[:,newaxis]) + lambda*I

# b = V_u^T (conf * w)
b = V_u.T @ (conf * w)

# Solve: A @ U[u] = b => U[u] = A^{-1} b
U[u] = np.linalg.solve(A, b)
```

5.6 Item Update Step (closed-form)

For each item i , with U fixed (same structure, swap roles):

```
U_i = U[user_indices_for_i]
conf = 1 + confidence_scale * w
A = U.T @ U + U_i.T @ (U_i * (conf-1)[:,newaxis]) + lambda*I
b = U_i.T @ (conf * w)
V[i] = np.linalg.solve(A, b)
```

5.7 Key Hyperparameters

n_factors	50 -- dimensionality of latent space. More = richer, but slower.
------------------	--

n_iteration

5.8 Prediction

```
# Single (user, trek) prediction
score = clip(U[user_i] @ V[trek_j], 0.5, 5.0)

# All treks for user (faster -- matrix multiply)
all_scores = clip(U[user_i] @ V.T, 0.5, 5.0)
```

Why implement ALS from scratch?

Demonstrates you understand the math, not just how to call a library. Real libraries (implicit, Surprise) use the same algorithm but are C-optimised. In a production system you would use the library version.

Section 6 -- Content-Based Filtering -- Deep Dive

6.1 What Is CBF?

Content-Based Filtering recommends items similar to what the user likes, based on item features -- without needing other users' data. It is especially useful when:

- A user is new (cold-start problem)
- There is not enough interaction history
- Items have rich descriptive features

6.2 The Process

1. Represent each trek as a 24-dimensional feature vector (normalised to [0,1]).
2. Build a user profile vector as the weighted average of features of treks the user interacted with.
3. For new users: map preferences directly to a feature vector.
4. Compute cosine similarity between the user vector and every trek vector.
5. Rank treks by similarity score.

6.3 Cosine Similarity

Cosine similarity measures the angle between two vectors, not their magnitude. It ranges from -1 (opposite) to +1 (identical direction). In practice with non-negative feature vectors, it ranges from 0 to 1.

```
# Weighted cosine similarity
A_w = A * weights    # apply feature importance weights
B_w = B * weights

similarity = (A_w @ B_w.T) / (||A_w|| * ||B_w||)
```

6.4 User Profile Vector for Known Users

```
# Get all treks this user interacted with + their weights
feature_matrix = trek_feature_scaled[trek_indices] # shape (n_interactions, 24)
weights        = interaction_weights              # shape (n_interactions,)

# Weighted mean of trek feature vectors
profile = (feature_matrix.T * weights).T.sum(axis=0) / weights.sum()
# result: 24-dimensional user preference vector
```

6.5 Preference-to-Vector for New Users (Cold Start)

```
# Start with median of all trek features as baseline
vec = median(trek_feature_scaled, axis=0)

# Override with user preferences
vec[difficulty_idx] = (pref.difficulty - 1) / 5.0
vec[fitness_idx]    = (pref.fitness - 1) / 3.0
vec[duration_days_idx] = clip((pref.duration_max - min) / (max - min), 0, 1)
vec[cost_min_idx]     = clip((pref.budget - cmin_min) / cmin_range, 0, 1)
```

```
vec[cost_max_idx] = clip((pref.budget - cmax_min) / cmax_range, 0, 1)
```

Section 7 -- Hybrid Blending Strategy

7.1 Why Hybrid?

Neither ALS nor CBF is perfect alone:

ALS alone: -- Cannot handle new users (cold-start)
-- Depends on volume of interactions to work well

CBF alone: -- "Filter bubble" -- only recommends similar treks
-- Cannot discover serendipitous recommendations

Combining them gets the benefits of both while mitigating the weaknesses.

7.2 Adaptive Alpha Blending

Both ALS and CBF scores are first normalised to [0, 5] using min-max normalisation, so they are on the same scale. Then they are blended:

```
final_score = alpha * ALS_normalised + (1 - alpha) * CBF_normalised

# Alpha is computed PER USER based on interaction history:
n_seen      = number of training interactions for this user
total_weight = sum of interaction weights
count_factor = min(0.85, n_seen / 20.0)
quality_factor = min(0.85, total_w / 30.0)
alpha       = clip(0.5 * count_factor + 0.5 * quality_factor, 0.10, 0.90)
```

Interpretation:

-- User with many high-quality interactions: alpha close to 0.90 ? ALS dominates
-- New / low-activity user: alpha close to 0.10 ? CBF dominates
-- Automatically adapts as the user gains more history

7.3 ALS Fold-In (Runtime Hybrid in app.py)

In the API, when a request provides interactions from a user not in the training set, the app uses "ALS fold-in": a fast approximation that computes a pseudo-user vector without re-training the model:

```
# For each interaction, take the trek's V vector, weighted by interaction weight
weighted_vecs = [als_V[trek_idx] * weight for trek_idx, weight in interactions]
total_weight  = sum(weights)
user_vec      = sum(weighted_vecs) / total_weight

# Then predict: user_vec @ als_V.T (dot product with all trek V vectors)
als_scores    = clip(user_vec @ als_V.T, 0.5, 5.0)
```

Section 8 -- Post-Ranking Re-Scorer

8.1 What Is It?

After ALS + CBF blending produces a ranked list, a lightweight re-scoring step applies small bonuses or penalties based on the enhanced metadata that was NOT included in the 24-feature matrix. This refines ordering without overriding the primary collaborative/content signal.

8.2 Bonus/Penalty Rules

- Season match: if trek season_rating ≥ 4.0 for user's preferred_season ? +0.10
- Season mismatch: if trek season_rating ≤ 2.0 ? -0.05
- AMS risk (user concern ≥ 4 AND trek AMS risk ≥ 3) ? -0.10 (bad match)
- AMS risk (user concern ≤ 2 AND trek AMS risk ≥ 3) ? +0.05 (they don't care)
- Accommodation match: user pref type in trek's available types ? +0.05
- Permit: user unwilling AND trek has restricted_area permit ? -0.05
- Safety: concerned user (ams ≥ 4) + heli evac available ? +0.03
- Safety: concerned user (ams ≥ 4) + ≥ 2 medical posts on route ? +0.02
- Total bonus clipped to [-0.30, +0.30]

Design Principle

The max bonus/penalty is ± 0.30 out of a 0-5 score range. This is deliberately small so it can only re-order treks that are otherwise very close in score. It cannot boost a poor match above a strong one.

Section 9 -- Evaluation Metrics

9.1 RMSE -- Root Mean Squared Error

Measures how accurately the model predicts interaction weights:

```
RMSE = sqrt( mean( (predicted - actual)^2 ) )

# Lower is better
# < 0.4 = Excellent
# < 0.6 = Good
# < 0.8 = Acceptable
# >= 0.8 = Needs tuning
```

RMSE penalises large errors more than small errors (because of the squaring). It is sensitive to outliers.

9.2 MAE -- Mean Absolute Error

Similar to RMSE but treats all errors equally:

```
MAE = mean( |predicted - actual| )
# Lower is better, typically slightly lower than RMSE
```

9.3 NDCG@K -- Normalised Discounted Cumulative Gain

A ranking metric that rewards putting relevant items near the TOP of the list. A relevant trek is defined as one with weight above the median in the test set.

```
# DCG = sum over positions 1..K of: relevance / log2(position + 1)
# Positions near top have small log2 denominator ? higher contribution

# IDCG = DCG if the list were perfectly ordered (ideal)
# NDCG = DCG / IDCG (normalised to 0?1)

# NDCG@10 measures: how good is the top-10 recommendation list?
```

NDCG@10 closer to 1.0 = better. This is the most important metric for ranking quality.

9.4 Precision@K and Recall@K

```
# hits = number of relevant treks in top-K recommendations
# relevant = test treks with weight > median_weight

Precision@K = hits / K
Recall@K     = hits / total_relevant_for_user
```

9.5 Why Use Multiple Metrics?

- RMSE/MAE: tells you how accurate the score prediction is
- NDCG: tells you whether the TOP recommendations are relevant (ranking quality)
- Precision@K: tells you what fraction of shown results are relevant (quality)
- Recall@K: tells you how many relevant results you are surfacing (coverage)

- A model can have good RMSE but poor NDCG if it predicts scores accurately but does not rank correctly
- both are needed

Section 10 -- train.py Step-by-Step Walk-through

train.py is the offline training pipeline. It runs once to build the model and save it. Here is what each step does:

STEP 1 -- Load & Clean

Reads dataset/final_trek_dataset_with_provinces.json. Fixes duplicate trek_id (puh001 ? puh002). Fills missing base_camp_altitude_m with the median value.

STEP 1b -- Synthesize Metadata

For each trek, calls 6 functions to generate enhanced metadata groups: _synthesize_seasonality, _synthesize_altitude_sickness, _synthesize_permits, _synthesize_accommodation, _synthesize_transportation, _synthesize_health_safety. These use existing trek fields (altitude, difficulty, food_avail etc.) plus controlled random noise. Stored in trek dicts but NOT added to the 24-feature matrix.

STEP 2 -- Encode Trek Features

Maps all categorical string fields to integers using lookup dicts. Builds a (188 x 24) numpy array. Scales to [0,1] with MinMaxScaler (fit on all treks). Computes variance-based feature importance weights.

STEP 3 -- Generate Users

600 users from 15 profiles x 40 per profile. Each user gets a preferences dict with 9 fields. Small random noise per user ensures variety within a profile.

STEP 4 -- Generate Interactions

45% density: for ~121,000 (user, trek) pairs, compute a base score via compute_base_score(). From base_score, derive views/booked/favorites/rating/time_spent_secs. Combine into a single weight. Min-max normalise to [0.5, 5.0].

STEP 5 -- Train/Test Split

80% train, 20% test using sklearn train_test_split(random_state=42). Build sparse CSR matrix from training interactions.

STEP 6 -- ALS Training

Hyperparameter grid search across 10 configs using a 15% validation split from training data. Select best config by val RMSE. Train final ALS model on full training data with 30 iterations.

STEP 7 -- Content-Based Filtering

Defines cosine_similarity_manual(), build_user_profile_vector(), preferences_to_vector(). No training needed -- CBF is computed on demand.

STEP 8 -- Hybrid System

Defines recommend_als(), recommend_cbf(), recommend_hybrid(), recommend_from_preferences(), recommend_for_new_user(). Adaptive alpha computed per user based on their interaction count and weight sum.

STEP 9 -- Evaluation

Computes test RMSE and MAE on ALS predictions for test interactions. Computes NDCG@10,

Precision@10, Recall@10 using the hybrid system on up to 200 users that have relevant test items.

STEP 10 -- Sample Output

Prints recommendations for 4 existing users and 4 new-user scenarios to visually verify quality.

STEP 11 -- Save Model

Saves everything to models/recommender.pkl using pickle. Includes: ALS matrices, trek features, user data, index mappings, interaction data, enhanced trek metadata, evaluation metrics.

STEP 12 -- Visualisation

Creates a 2x3 matplotlib figure: Training RMSE curve, Predicted vs Actual scatter, Interaction weight distribution, Top recommended treks, Season suitability bar chart, AMS Risk vs Accommodation quality scatter.

Section 11 -- app.py Flask API Walk-through

11.1 Startup

On startup, app.py loads models/recommender.pkl and unpacks all artefacts into module-level variables. This means the heavy data is in memory for the lifetime of the process -- no reload per request.

11.2 API Endpoints

POST /recommend

Pure CBF -- no interactions needed. Takes a preferences dict in JSON body. Returns top_n treks by cosine similarity + penalty + rerank bonus. Good for first-time users or simple search.

POST /recommend/hybrid

Main endpoint. Takes preferences + optional interactions array. If no interactions ? pure CBF (cold start). If interactions ? ALS fold-in + CBF blend. Returns recommendations with metadata including model_used, alpha, interaction_count, skipped_interactions.

POST /recommend/als

ALS for training users (looks up user_id in user_index). Falls back to CBF if user_id not found. Mostly for testing / legacy support.

GET /recommend/user/<user_id>

Full hybrid for a training user -- uses pre-trained U matrix. Returns recommendations + user preferences + interaction stats. Useful for testing known users from training data.

GET /health

Health check. Returns: status, treks_loaded, known_users, als_factors, test_rmse, test_mae.

GET /treks/count

Simple count of treks in the loaded model.

GET /users

Returns list of all user_ids from training data.

11.3 Key Functions in app.py

compute_weight()

Converts raw interaction signals to a single weight (mirrors train.py formula)

validate_in

11.4 The ALS Fold-In Explained

Standard ALS requires retraining the U matrix when a new user arrives. Fold-in is a fast alternative: given a user's interaction history, estimate their U vector without touching the existing U/V matrices:

```
# Each interaction contributes: V[trek_idx] weighted by interaction weight
user_vec = sum(V[trek_idx] * weight for trek_idx, weight) / sum(weights)

# Predict scores: dot product with all item vectors
als_scores = clip(user_vec @ V.T, 0.5, 5.0)
```

This is an approximation (not the exact ALS solution) but it is instant and works well in practice for users with a few interactions.

Section 12 -- The Cold-Start Problem & How It Is Solved

12.1 What Is the Cold-Start Problem?

The cold-start problem occurs when a recommendation system has no prior interaction data for a user or item. This is a fundamental challenge because collaborative filtering requires interaction history to work.

12.2 Two Variants

- User cold-start: A new user arrives with no interaction history. ALS has no row in U for them ? cannot compute $U[u] @ V.T$.
- Item cold-start: A new trek is added with no interactions. ALS has no row in V for it ? cannot score it against users.

12.3 How This System Solves User Cold-Start

1. If NO interactions are provided ? use pure CBF with `preferences_to_vector()`. Builds a 24-dim vector from user preferences and computes cosine similarity.
2. If SOME interactions are provided ? ALS fold-in: estimates user vector from item vectors of interacted treks, weighted by interaction weight. Then blends with CBF using fixed alpha (0.95 if ≥ 5 interactions, 0.90 otherwise).
3. Post-ranking bonus further personalises using season, AMS concern, accommodation.

12.4 Why CBF Is Good for Cold-Start

CBF only needs the user's stated preferences (difficulty, budget, etc.) to generate meaningful recommendations immediately. It does not need interaction history. The trade-off is the "filter bubble" -- it will recommend treks very similar to stated preferences without surprise discoveries.

Section 13 -- Hyperparameter Tuning

13.1 What Is Tuned?

The ALS model has 4 hyperparameters. The training script runs a grid search across 10 pre-defined configurations on a 15% validation split of training data. The config with lowest validation RMSE wins.

13.2 The Grid

- n_factors=25, n_iterations=20, lambda=0.10, conf_scale=1.0
- n_factors=25, n_iterations=20, lambda=0.10, conf_scale=5.0
- n_factors=50, n_iterations=20, lambda=0.05, conf_scale=1.0
- n_factors=50, n_iterations=20, lambda=0.05, conf_scale=5.0 ? often best
- n_factors=50, n_iterations=20, lambda=0.10, conf_scale=5.0
- n_factors=75, n_iterations=20, lambda=0.05, conf_scale=1.0
- n_factors=75, n_iterations=20, lambda=0.05, conf_scale=5.0
- n_factors=50, n_iterations=20, lambda=0.05, conf_scale=10.0
- n_factors=50, n_iterations=20, lambda=0.02, conf_scale=1.0
- n_factors=50, n_iterations=20, lambda=0.02, conf_scale=5.0

After grid search, the best config is retrained for 30 iterations (instead of 20) on the FULL training set (not just the 85% sub-split).

13.3 Effect of Each Hyperparameter

n_factors ?	More expressive model; slower training; risk of overfitting on small data	n_factors
-------------	---	-----------

Section 14 -- Things to Know & Learn

14.1 Core Concepts You Must Be Able to Explain

- What is Matrix Factorization and why is it used for recommendations?
- What does the latent factor space represent?
- What is the difference between explicit and implicit feedback?
- What is the ALS objective function and how is it minimised?
- Why does ALS alternate between user and item updates?
- What is cosine similarity and why is it preferred over Euclidean distance for feature vectors?
- What is the cold-start problem and how do you solve it?
- What is adaptive alpha blending and why does it help?
- What does NDCG measure and why is it better than accuracy for ranking?
- Why use confidence-weighted ALS for implicit feedback?

14.2 Python Libraries Used

- numpy -- matrix operations, linear algebra (linalg.solve), random seed
- pandas -- DataFrame for interaction data, groupby, train_test_split input
- scipy.sparse -- CSR/CSC sparse matrices for efficient ALS computation
- sklearn -- MinMaxScaler for feature normalisation, train_test_split
- matplotlib -- visualisation of training curves and distributions
- flask -- REST API server, JSON request/response handling
- pickle -- serialise/deserialise the trained model artefacts

14.3 Key Python Patterns to Understand

- `np.linalg.solve(A, b)` -- solves $A @ x = b$, more stable than $A^{-1} @ b$
- `csr_matrix.getrow(u).indices` -- gets non-zero column indices for row *u* (sparse)
- `np.argsort(scores)[::-1][:k]` -- get indices of top-k scores efficiently
- `np.clip(arr, lo, hi)` -- clamp array values to a range
- `MinMaxScaler.fit_transform()` -- fit parameters on training data, transform immediately
- `@app.route(path, methods=[...])` -- Flask decorator for defining API endpoints
- `request.get_json()` -- parse incoming JSON in Flask
- `jsonify()` -- convert Python dict to Flask JSON response

14.4 Mathematical Prerequisites

- Linear algebra: matrix multiplication, transpose, dot product
- Vector norms: $\|v\| = \sqrt{\sum(v_i^2)}$
- Least squares: solving $A @ x = b$ via normal equations
- L2 regularisation: adding $\lambda * I$ to the *A* matrix prevents singularity and overfitting
- Loss functions: MSE, RMSE, MAE -- when to use which
- Ranking metrics: DCG, NDCG -- logarithmic discount for position
- Normalisation: Min-Max scaling to $[0,1]$ or $[lo, hi]$

14.5 What to Study Next (Improvements)

- Implicit library (pip install implicit) -- C-optimised ALS, GPU support
- Real user feedback loop -- replace synthetic interactions with real data
- Neural Collaborative Filtering (NCF) -- uses deep learning instead of dot product
- BERT4Rec / Transformer-based sequential recommendations
- Item cold-start -- add new treks without retraining (use content features only)
- A/B testing framework -- compare model versions with real users
- Multi-armed bandit exploration -- balance exploitation vs. discovering new preferences
- User feedback collection -- explicit ratings via app UI

Section 15 -- Interview Questions & Model Answers

Q1: What is the difference between collaborative filtering and content-based filtering?

Collaborative filtering (CF) finds patterns from user behaviour -- "users like you also liked this". It does not need item features, only interaction data. Content-based filtering (CBF) recommends items similar to ones the user liked, based on item features. CF requires interaction history; CBF requires item descriptions. CF discovers serendipitous items; CBF tends toward a "filter bubble". This system uses both in a hybrid.

Q2: Why did you implement ALS from scratch instead of using a library?

To demonstrate understanding of the underlying mathematics. The from-scratch implementation shows the confidence-weighting formula, the alternating least-squares solve (`np.linalg.solve`), and the RMSE tracking. In production I would use the "implicit" library which is C-optimised and ~100x faster for large datasets.

Q3: What is the cold-start problem and how does your system handle it?

Cold-start occurs when a new user has no interaction history. ALS cannot produce recommendations without a user row in the U matrix. This system solves it with two mechanisms: (1) pure CBF using stated user preferences mapped to a 24-dim feature vector when no interactions exist; (2) ALS "fold-in" -- when a few interactions are provided, estimate the user vector as a weighted average of the interacted items' V vectors, then blend with CBF using a high alpha.

Q4: What does NDCG measure and why is it better than accuracy?

NDCG -- Normalised Discounted Cumulative Gain -- measures the quality of the RANKING of recommendations, not just whether items are relevant. It gives more credit for putting relevant items near the TOP of the list (position 1 contributes more than position 10, via a log2 discount). NDCG@10 near 1.0 means the top-10 list is close to perfect ordering. Accuracy only asks "is it correct?" -- NDCG asks "is the most relevant item shown first?".

Q5: Why normalise ALS and CBF scores before blending?

ALS scores are in the range [0.5, 5.0] (interaction weights). CBF scores are cosine similarities in [-1, 1] or [0, 1]. If blended directly, ALS would dominate due to larger scale. Min-max normalising both to [0, 5] puts them on equal footing before the alpha-weighted blend.

Q6: What does the regularisation term (lambda) do in ALS?

The $\lambda * I$ term is added to matrix A in each least-squares solve: $A = V^T V + V_u^T \text{diag}(\text{conf}-1) V_u + \lambda * I$. This is L2 regularisation (ridge regression). It prevents the matrices U and V from growing too large, which reduces overfitting. It also ensures A is always invertible (positive definite), which prevents numerical issues when some rows of V_u are near-zero.

Q7: How do you choose the interaction density (45%)? What would happen at 10% vs 90%?

The 45% density is a design choice for a realistic simulation. At 10% density the matrix would be very sparse -- ALS would have fewer signals per user and item, leading to higher RMSE and poorer recommendations. At 90% density the matrix is very dense -- ALS would have abundant signal but the dataset would be unrealistically complete (real systems have <5% density). The 45% was chosen to balance model quality with simulation realism.

Q8: What is the purpose of the confidence_scale hyperparameter?

confidence_scale controls how strongly high-weight interactions pull the user/item embeddings. With confidence_scale=5.0, a booked trek (weight ~5.0) has confidence $c_{ui} = 1 + 5 \cdot 5 = 26$, while a viewed-only trek (weight ~0.5) has $c_{ui} = 1 + 5 \cdot 0.5 = 3.5$. The model is 7x more penalised for mispredicting the booked trek. This makes the model respect strong signals more.

Q9: Why not use cosine similarity directly on raw (unscaled) features?

Raw features are on very different scales: duration_days might be 5-25, max_altitude_m might be 1000-8848. Without normalisation, high-magnitude features like altitude would dominate the cosine computation. Min-max scaling to [0,1] puts all features on equal footing before applying the feature-importance weights, which then correctly re-weight by variance.

Q10: What is ALS fold-in and what are its limitations?

ALS fold-in is a fast way to produce recommendations for a new user without retraining. It estimates the user's latent vector as a weighted average of the item (trek) vectors for their interacted items. Limitations: it is an approximation -- the true ALS solution for a new user would solve the full least-squares problem considering all other users. Fold-in also does not update the item vectors, so if the new user has unusual taste, the item vectors may not represent them well. It works best with at least 3-5 interactions.

Q11: Why use synthetic data? What are the risks?

Synthetic data is used because no real Nepal trek interaction dataset exists publicly. It allows demonstration of the full pipeline. Risks: the synthetic interactions are generated by the same scoring function the model tries to learn -- this makes the task artificially easy (the model is essentially learning its own data generator). Real-world performance would depend on actual user behaviour which may be much noisier and more complex.

Q12: How would you deploy this system for real users?

The Flask API (app.py) is the deployment artifact. It loads the model once at startup. For each request, it runs CBF or hybrid inference (milliseconds). For production: (1) use gunicorn (already in requirements) with multiple workers; (2) cache the loaded model in shared memory; (3) add periodic retraining as new interaction data accumulates; (4) use a proper database for interactions instead of in-memory data; (5) add authentication, rate limiting, and logging.

Quick Reference Cheat Sheet

ALS at a Glance

```
R (n_users x n_treks) ? U (n_users x k) @ V.T (k x n_treks)

c_ui = 1 + conf_scale * w_ui          # confidence weight
Loss = sum c_ui*(w_ui - U[u]@V[i])^2 + lambda*(||U||^2+||V||^2)

User update:  A = V.T@V + V_u.T@diag(c-1)V_u + lam*I
              b = V_u.T @ (c*w)
              U[u] = solve(A, b)

Item update:  (same, swap U <-> V, users <-> items)

Predict:      score(u,i) = clip(U[u] @ V[i], 0.5, 5.0)
```

Hybrid Score Formula

```
final = alpha * norm(ALS) + (1-alpha) * norm(CBF)
        - difficulty_fitness_penalty
        + rerank_bonus      # in [-0.30, +0.30]

alpha = clip( 0.5*(n_seen/20) + 0.5*(weight_sum/30), 0.10, 0.90 )
```

Metric Formulas

```
RMSE  = sqrt( mean( (pred - actual)^2 ) )
MAE   = mean( |pred - actual| )
DCG@K = sum_{p=1}^K rel_p / log2(p+1)
NDCG@K = DCG@K / IDCG@K      (IDCG = ideal DCG)
Prec@K = hits / K
Rec@K  = hits / total_relevant
```

API Endpoints Summary

```
POST /recommend      -> pure CBF (cold start)
POST /recommend/hybrid -> CBF or ALS fold-in + CBF (adaptive)
POST /recommend/als   -> ALS for known user, CBF fallback
GET  /recommend/user/<id>-> full hybrid for training user
GET  /health          -> status + metrics
```